# Solving Air Crew Scheduling Problems using Simulated Annealing and Genetic Algorithms

## 1. Algorithm Descriptions

### 1.1 Simulated Annealing (SA)

SA is a single-solution metaheuristic that explores the search space by iteratively perturbing the current solution and accepting or rejecting changes based on a temperature-controlled probabilistic criterion.

**Key concepts:**

- Binary representation: each bit indicates whether a column (crew pairing) is selected
- Initialization: `feasible_init` retries pseudo-random init (Algorithm 2) until a fully feasible solution is produced
- Neighbor: 50/50 mix of single bit flips and swap moves (remove a selected column, add an unselected column sharing a covered row)
- Constraint handling: penalty ramp. Lambda grows from 0.5*max(costs) to max(costs) over each restart
- Cooling: geometric schedule with alpha computed to reach T_final=0.1 from T0=5*mean(costs) over iter_per_run iterations
- Multi-restart: 10 restarts, each with 50,000 iterations (500,000 total), keeping the global best
- Best tracking: only records feasible solutions (violation count = 0) as best

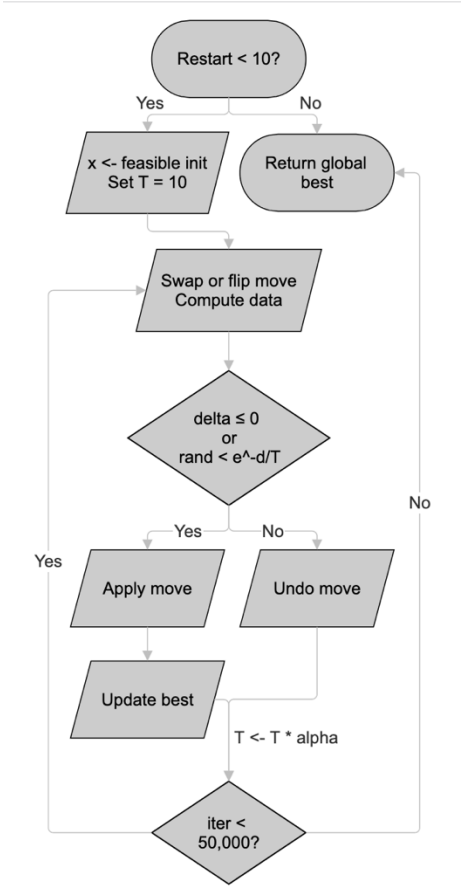


*Figure 1*

**Pseudocode - Simulated Annealing:**

```
best_x ← None, best_cost ← ∞

FOR restart = 1 TO 10:
    x ← feasible_init()
    T ← T0 = 5·mean(costs)
    alpha ← (0.1 / T0)^(1/50000)
    run_best ← cost(x) if feasible else ∞

    FOR iter = 1 TO 50,000:
        lam ← max(costs) · (0.5 + 0.5 · iter/max_iter)

        WITH probability 0.5:
```

```
            // swap move: remove random selected col, add random col
sharing a row
        ELSE:
            // single bit flip on random column

        delta ← new_penalized_cost - old_penalized_cost
        IF delta ≤ 0 OR rand() < exp(-delta / T):
            ACCEPT move
            IF feasible AND cost < run_best:
                UPDATE run_best
        ELSE:
            UNDO move

        T ← T · alpha

    IF run_best < best_cost:
        UPDATE best

RETURN best_x, best_cost
```

## 1.2 Standard Binary Genetic Algorithm (BGA)

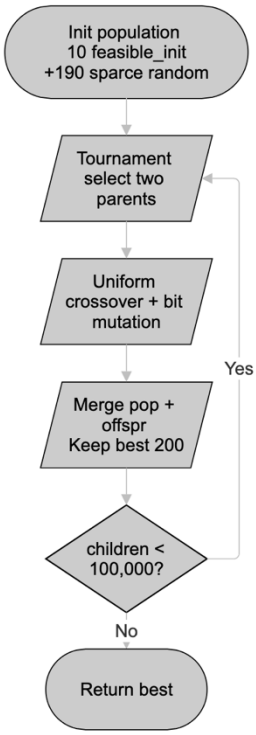

*Figure 2*

A population-based algorithm using penalized fitness for constraint handling, tournament selection, uniform crossover, and truncation replacement.

**Key design choices:**

- Population size: 200. 10 individuals seeded via `feasible_init` (guaranteed feasible), rest are sparse random vectors (density ~ 2m/n)
- Selection: binary tournament on penalized fitness
- Crossover: uniform crossover (each bit independently from either parent with equal probability)
- Mutation: flip one random bit per child
- Constraint handling: penalty function. penalized_cost = raw_cost + max(costs) * violation_count
- Replacement: generational with truncation. merge parents and offspring, keep best pop_size by penalized fitness

**Pseudocode - Standard BGA:**

```
pop ← 10 feasible_init() + 190 sparse random binary
vectors
fitnesses ← [penalized_cost(x) for x in pop]
best_x ← None, best_cost ← ∞
children_count ← 0
```

```
WHILE children_count < 100,000:
    FOR each generation batch:
        p1 ← tournament_select(pop, k=2)
        p2 ← tournament_select(pop, k=2)
        child ← uniform_crossover(p1, p2)
        single_bit_mutation(child)
        child_fit ← penalized_cost(child)
        IF child is feasible AND cost < best_cost:
            UPDATE best
    MERGE pop + offspring, KEEP best 200 by penalized fitness
    children_count += batch_size

RETURN best_x, best_cost
```

## 1.3 Improved BGA (Chu & Beasley)

An enhanced genetic algorithm incorporating three key improvements from the literature: pseudo-random initialization, heuristic improvement operator applied to every child, and stochastic ranking with ranking replacement for constraint handling.

**Key design choices:**

- Population size: 100, initialized via Algorithm 2 (pseudo-random init), individuals may have under-covered rows
- Selection: stochastic ranking (Runarsson & Yao) with P_f = 0.45 determines parent ranking; parents drawn from top half
- Crossover: uniform crossover producing one child
- Mutation: adaptive. Flip k=5 bits if duplicates exist in population, else k=3 bits
- Repair: heuristic improvement (Algorithm 1) applied to every child, eliminating over-coverage (under-coverage handled by stochastic ranking)
- Replacement: ranking replacement - deterministic lexicographic comparison with worst individual based on violation and fitness
- Termination: 100,000 non-duplicate children generated

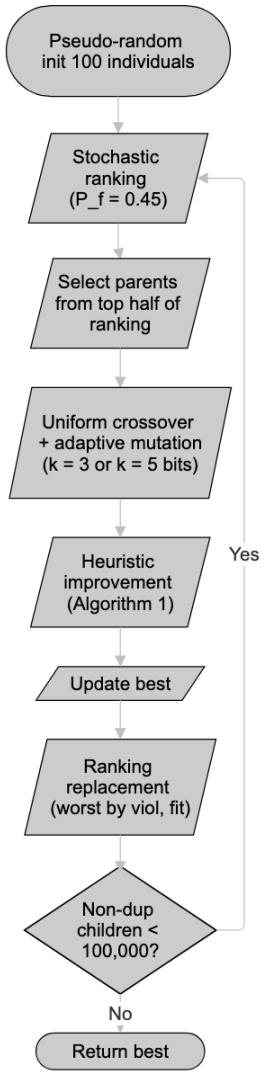


*Figure 3*

**Algorithm 1 — Heuristic Improvement Operator (p331, [1]):**

```
FUNCTION heuristic_improvement(x):
    cover ← A · x    (row coverage counts)

    // DROP phase: inspect selected columns in random
order
    FOR j IN selected columns (shuffled):
        IF any row covered by j has cover ≥ 2:
            x[j] ← 0, update cover

    // ADD phase: visit uncovered rows in random order, each row once
    FOR i IN uncovered rows (shuffled):
        IF cover[i] ≠ 0: SKIP
```

```
        FOR each unselected column j covering row i:
            IF all rows of j are uncovered (β_j ⊆ U):
                score[j] ← cost[j] / |rows covered by j|
        j* ← argmin score
        x[j*] ← 1, update cover

    RETURN x   (no over-coverage; may have under-covered rows)
```

Note: the ADD phase only adds columns whose *all* rows are currently uncovered (β_j ⊆ U constraint), which prevents introducing over-coverage but may leave some rows uncovered. This is by design — the improved BGA handles residual infeasibility via stochastic ranking. SA and standard BGA use `feasible_init`, which retries `pseudo_random_init` until a fully feasible solution is obtained.

**Algorithm 2 — Pseudo-Random Initialization:**

```
FUNCTION pseudo_random_init():
    x ← all zeros
    uncovered ← {all rows}
    WHILE uncovered not empty:
        i ← random row from uncovered
        j ← random column covering row i
        x[j] ← 1
        uncovered ← uncovered - {rows covered by j}
    RETURN heuristic_improvement(x)
```

**Pseudocode: Improved BGA**

```
pop ← [pseudo_random_init() for _ in range(100)]
fitnesses ← [raw_cost(x) for x in pop]
violations ← [count uncovered rows for x in pop]
children_count ← 0

WHILE children_count < 100,000:
    k ← 5 if duplicates exist else 3

    ranked ← stochastic_ranking(fitnesses, violations, pf=0.45)
    p1 ← pop[ranked[rand(0, N/2)]]
    p2 ← pop[ranked[rand(0, N/2)]]

    child ← uniform_crossover(p1, p2)
    flip k random bits in child
    child ← heuristic_improvement(child)
```

```
    IF child is feasible AND cost < best_cost:
        UPDATE best

    ranking_replace(pop, child)   // replace worst by (violation,
fitness)

    IF child is not duplicate:
        children_count += 1

RETURN best_x, best_cost
```

**Stochastic Ranking (Runarsson & Yao [2]):**

```
FUNCTION stochastic_ranking(pop, fitnesses, violations, pf=0.45):
    indices ← [0, 1, ..., N-1]
    REPEAT:
        swapped ← false
        FOR i = 0 TO N-2:
            IF (violations[indices[i]] == 0 AND
violations[indices[i+1]] == 0) OR rand() < pf:
                // compare by fitness only
                IF fitnesses[indices[i]] > fitnesses[indices[i+1]]:
                    SWAP indices[i], indices[i+1]
                    swapped ← true
            ELSE:
                // compare by violation
                IF violations[indices[i]] > violations[indices[i+1]]:
                    SWAP indices[i], indices[i+1]
                    swapped ← true
    UNTIL not swapped
    RETURN indices
```

# 2. Results

Results from 30 independent runs per algorithm per problem. Seeds: `random.seed(42 + run)`, `np.random.seed(42 + run)` for run = 0..29.

| *Algorithm* | *Mean* | *Std Dev* | *Best* | *Gap (%)* | *Avg Time (s)* |
|---|---|---|---|---|---|
| *SA* | 12924.1 | 859.9 | 11307 | 0.00% | 2.4 |
| *Standard BGA* | 13732.1 | 1609.1 | 11307 | 0.00% | 1.3 |
| *Improved BGA* | 11307.0 | 0.0 | 11307 | 0.00% | 6.0 |

*Table 1: sppnw41 (17 rows, 197 columns, optimal = 11307)*

| *Algorithm* | *Mean* | *Std Dev* | *Best* | *Gap (%)* | *Avg Time (s)* |
|---|---|---|---|---|---|
| *SA* | 9115.3 | 853.8 | 7674 | 0.24% | 5.9 |
| *Standard BGA* | 9491.2 | 966.6 | 7656 | 0.00% | 2.9 |
| *Improved BGA* | 7658.0 | 4.1 | 7656 | 0.00% | 31.7 |

*Table 2: sppnw42 (23 rows, 1079 columns, optimal = 7656)*

| *Algorithm* | *Mean* | *Std Dev* | *Best* | *Gap (%)* | *Avg Time (s)* |
|---|---|---|---|---|---|
| *SA* | 11010.5 | 802.8 | 9570 | 7.48% | 5.6 |
| *Standard BGA* | 11000.0 | 765.9 | 8904 | 0.00% | 2.5 |
| *Improved BGA* | 8904.0 | 0.0 | 8904 | 0.00% | 26.6 |

*Table 3: sppnw43 (18 rows, 1072 columns, optimal = 8904)*

# 3. Comparison and Discussion

## Performance Ranking

The improved BGA dominates: it finds the integer optimum on every single run for sppnw41 and sppnw43 (std=0), and achieves mean=7658 on sppnw42 (0.03% above optimal, std=4.1). SA and standard BGA show similar mean performance to each other (means 20-24% above optimal) but with high variance (std 800-1600). Both SA and standard BGA occasionally find the optimum on lucky runs, but cannot do so reliably.

## Why the Improved BGA Works Best

**Heuristic improvement.** Algorithm 1 eliminates over-coverage from every child, dramatically reducing the feasible search space. Even when it leaves some rows under-covered, stochastic ranking handles the residual infeasibility gracefully. This is far more effective than penalty functions, which give SA and standard BGA no structural guidance toward feasibility.

**Intelligent initialization.** Pseudo-random initialization (Algorithm 2) produces starting solutions that are already close to feasible set partitions. By contrast, sparse random initialization in the standard BGA creates mostly infeasible individuals, requiring many generations just to find feasibility.

**Adaptive mutation.** The improved BGA flips k=5 bits when duplicates exist in the population, preserving diversity. When the population is diverse, k=3 bits performs fine-grained local search. This prevents the premature convergence that plagues the standard BGA (std=1609 on sppnw41, indicating it frequently gets stuck).

**Effective selection pressure.** Stochastic ranking (P_f=0.45) occasionally favors fit infeasible individuals during parent selection, maintaining exploration. Ranking replacement ensures population quality never degrades. This combination produces steady convergence without population collapse.

## SA vs Standard BGA Tradeoffs

SA with multi-restart (10 × 50,000 iterations) and swap moves achieves comparable means to the standard BGA across all problems. On sppnw42, SA actually outperforms standard BGA on mean (9115 vs 9491). SA is conceptually simpler and faster per iteration, but its single-solution trajectory limits exploitation of good regions.

The standard BGA benefits from population diversity but wastes effort evaluating infeasible solutions. Its penalty function (lam = max(costs)) gives coarse feasibility pressure. With 200 individuals and truncation replacement, it explores broadly but converges slowly.

## Scalability

On the larger problems (sppnw42/43: ~1079 columns), the improved BGA takes 27-32s per run, roughly 5x longer than SA and 10x longer than standard BGA. This cost is entirely due to the heuristic improvement operator applied to every child. Despite the per-evaluation cost, the improved BGA produces vastly better solutions: mean gap <0.03% vs 19-24% for the other algorithms.

# 4. Ranking Replacement vs Stochastic Ranking

Both ranking replacement (Chu & Beasley [1]) and stochastic ranking (Runarsson & Yao [2]) are constraint handling techniques that avoid explicit penalty functions. They are used for different purposes in the improved BGA and operate on different principles.

## Ranking Replacement [1]

Used for **population management** (deciding which individual to replace when inserting a new child).

- **Deterministic**: always ranks feasible solutions above infeasible ones
- **Lexicographic ordering**: compares individuals by violation count, fitness. Lower violation is always better; among equal violations, lower cost wins
- **Replacement rule**: find the worst-ranked individual; replace it with the child only if the child ranks strictly better
- **No parameters**

This provides strong selection pressure toward feasibility and cost reduction. Every replacement either maintains or improves the population quality.

## Stochastic Ranking [2]

Used for **parent selection** (ranking the population to decide which individuals get to reproduce).

- **Probabilistic**: with probability $P_f$ (= 0.45), compares two adjacent individuals by fitness alone, regardless of feasibility
- **Bubble-sort mechanism**: iterates over the population, swapping adjacent individuals based on the probabilistic comparison rule, until no more swaps occur
- **Key parameter**: $P_f$ controls the tradeoff between fitness pressure and feasibility pressure
- **Effect**: feasible solutions tend to rank higher, but ~45% of the time an infeasible individual with good fitness can outrank a feasible one

## Similarities

- Both avoid explicit penalty functions and their associated parameter tuning (penalty weight)
- Both provide a mechanism to rank population members considering both fitness and constraint satisfaction
- Both are applied within a steady-state GA framework
- Both handle the fundamental tension between exploring infeasible but promising regions and exploiting feasible solutions

## Differences

| *Aspect* | ***Ranking Replacement*** | ***Stochastic Ranking*** |
|---|---|---|
| *Purpose* | Replacement decision | Parent selection ranking |
| *Nature* | Deterministic | Probabilistic |
| *Feasibility priority* | Always: feasible > infeasible | Probabilistic (P_f chance to ignore) |
| *Parameters* | None | P_f ≈ 0.45 |
| *Diversity effect* | Lower — strictly enforces quality | Higher — allows infeasible survival |
| *When applied* | Once per child | Once per generation |

The combination of both techniques in the improved BGA is effective: stochastic ranking maintains diversity during parent selection by occasionally favoring fit-but-infeasible individuals, while ranking replacement ensures the population quality monotonically improves by deterministically replacing the worst member. This pairing balances exploration and exploitation.